

UT1: Manejo de Ficheros (Texto, Binarios y XML)

Manual Técnico y Didáctico de Persistencia Física, Flujos I/O e Intercambio de Datos en Java

MÓDULO FORMATIVO	UNIDAD DE TRABAJO	RESULTADOS DE APRENDIZAJE
Acceso a Datos (0486)	UT1 / UD01 (30 Horas)	RA1 (CE1.a, CE1.b, CE1.c, CE1.d, CE1.e)
PROFESOR Sergio Capdevila Díez (Curso 2026-2027)		

Estructura Didáctica de la Unidad

- | | |
|--|--|
| 1. 1. Introducción Contextualizada | 2. 2. Conceptos Fundamentales de I/O |
| 3. 3. Desarrollo Progresivo de Contenidos | 4. 4. Ejemplos Prácticos Graduados |
| 5. 5. Ejemplos de Código Funcional Paso a Paso | 6. 6. Errores Habituales y Soluciones Técnicas |
| 7. 7. Buenas Prácticas de la Industria | 8. 8. Matrices Comparativas de Tecnologías |
| 9. 9. Resumen Clave de la Unidad | 10. 10. Conceptos que Debes Dominar |
| 11. 11. Glosario y Preguntas de Repaso | |

1 Introducción Contextualizada

En cualquier sistema de software profesional, los datos almacenados en variables o estructuras de memoria RAM desaparecen en cuanto la aplicación finaliza o el servidor se reinicia. Para garantizar que la información sobreviva en el tiempo, es imprescindible implementar mecanismos de **persistencia** en medios no volátiles (discos duros HDD, unidades SSD o sistemas de almacenamiento en red).

Utilidad en el Mundo Profesional

- Aunque en proyectos avanzados la persistencia principal recae en motores de bases de datos relacionales o NoSQL, el manejo de ficheros sigue siendo el núcleo insustituible para:
- **Registros del sistema (Logs):** Auditoría y monitorización continua de eventos en servidores de aplicaciones.
 - **Ficheros de configuración:** Parámetros de arranque en formatos `.properties`, `.env` o `.yaml`.
 - **Exportación masiva e ingesta ETL:** Migración e intercambio de información mediante ficheros tabulares CSV.
 - **Intercambio B2B y facturación electrónica:** Facturae (XML) e integración interbancaria.

¿Qué conocimientos previos necesitas? Para abordar con éxito esta unidad únicamente requieres conocimientos consolidados de Programación en Java de 1º curso: orientación a objetos (clases, constructores y métodos), uso de colecciones básicas (`List`, `Map`) y gestión estructurada de excepciones mediante bloques `try-catch`.

2 Conceptos Fundamentales de I/O

2.1. El Concepto Intuitivo de Flujo (Stream)

Imagina una tubería de agua que conecta un depósito con tu casa. Tú no recibes todo el depósito de golpe en un único instante; el agua fluye de manera continua gota a gota o litro a litro a través de la tubería.

En informática, un **Stream (Flujo)** funciona con esa misma metáfora: es un canal de comunicación unidireccional por el que viaja una secuencia ordenada de datos entre tu programa Java y un recurso externo (un fichero en disco, un puerto de red o la consola). Si el flujo va hacia el programa, hablamos de un **flujo de entrada (Input / Read)**; si los datos salen desde el programa hacia el almacenamiento, es un **flujo de salida (Output / Write)**.

💡 Idea Clave: Dos Familias de Flujos en Java

Java divide estrictamente sus flujos de entrada/salida en dos jerarquías principales:

- Flujos de Bytes (8 bits):** Clases base `InputStream` y `OutputStream`. Diseñadas para datos binarios puros (imágenes, audio, ficheros compilados o tipos de datos primitivos).
- Flujos de Caracteres (Unicode):** Clases base `Reader` y `Writer`. Diseñadas específicamente para texto humano, gestionando automáticamente la traducción entre caracteres y bytes según un juego de caracteres.

2.2. Juegos de Caracteres (Charsets) y el Problema del Mojibake

En la memoria RAM, Java almacena los caracteres usando internamente **UTF-16** (2 bytes por carácter). Sin embargo, al guardar texto en disco o enviarlo por la red, esos caracteres deben traducirse a secuencias de bytes según una tabla de equivalencias llamada **Charset**.

El estándar indiscutible de la industria global es **UTF-8** (1 byte para caracteres ingleses ASCII y de 2 a 4 bytes para tildes, eñes y caracteres internacionales). Si guardas un fichero en Windows asumiendo el juego por defecto (habitualmente `Windows-1252` o `ISO-8859-1`) y lo lees en un servidor Linux configurado en UTF-8 sin especificar el Charset, se producirá **Mojibake**: la corrupción visual de caracteres (ej. "CanciÃ³n" en vez de "Canción").

3 Desarrollo Progresivo de Contenidos

Avanzaremos a lo largo de 4 modelos tecnológicos de persistencia en disco, desde el texto plano hasta los lenguajes estructurados de intercambio.

3.1. Ficheros de Texto Plano: java.io Tradicional vs Java NIO.2

En el paquete clásico `java.io`, para trabajar con texto usamos `FileReader` y `FileWriter`. Sin embargo, acceder directamente al disco por cada carácter individual resulta extremadamente lento. Para solucionarlo, aplicamos el **Patrón Decorador** envolviendo el flujo con un búfer de memoria intermedia:

- `BufferedReader`: Almacena fragmentos del fichero en memoria RAM y ofrece el método `readLine()`, que recupera una línea completa de texto de forma eficiente.

- **BufferedWriter** : Retiene las escrituras en memoria hasta llenar un bloque y ofrece `newLine()` , garantizando el salto de línea adecuado al sistema operativo subyacente.
- **PrintWriter** : Extiende las capacidades de escritura facilitando métodos de formateo avanzado como `printf()` y `println()` sin lanzar excepciones comprobadas.

⚡ La API Moderna: Java NIO.2 (java.nio.file)

A partir de Java 7 y consolidada en las versiones modernas (Java 17 / 21 LTS), la API **NIO.2** sustituye al clásico **File** por dos abstracciones superiores:

- **java.nio.file.Path** : Representación inmutable de una ruta en el sistema de archivos (ej. `Path.of("datos", "ventas.csv")`).
- **java.nio.file.Files** : Clase de utilidad estática que permite leer o escribir ficheros pequeños en una sola línea (`Files.readString()` , `Files.writeString()`) o procesar ficheros masivos de millones de líneas sin saturar la RAM mediante streams reactivos evaluados de forma perezosa (`Files.lines()`).

💻 Trabajo Síncrono en Aula: Actividades 1.1 a 1.5

Abre tu IDE de clase (Eclipse) y realiza las actividades **1.1 (Conversión de Charsets)**, **1.2 (Normalización con Buffers)** y **1.5 (Procesamiento de Logs masivos con NIO.2)** detalladas en el [Cuaderno de Actividades de Aula](#).

3.2. Ficheros Binarios de Tipos Simples y Acceso Aleatorio (RandomAccessFile)

Cuando no necesitamos que los datos sean legibles por humanos en un bloc de notas, el formato binario aporta dos ventajas imbatibles: **mínimo tamaño en disco** y **velocidad extrema**.

- **DataOutputStream** y **DataInputStream** : Permiten escribir y leer directamente tipos de datos primitivos de Java (`writeInt()` , `writeDouble()` , `writeUTF()`) en formato binario estandarizado e independiente de la arquitectura del procesador.
- **RandomAccessFile (RAF)**: A diferencia de los streams secuenciales (donde para leer el byte 1.000 debes pasar obligatoriamente por los 999 anteriores), **RandomAccessFile** se comporta como un gran array de bytes persistido en disco. Dispone de un **puntero de lectura/escritura** que podemos mover libremente a cualquier posición con `seek(long posicion)` .

💡 El Secreto del Acceso Aleatorio: Registros de Longitud Fija

Para que `seek()` pueda saltar de forma instantánea a un registro concreto sin recorrer el fichero, **todos los registros deben ocupar exactamente el mismo número de bytes**.

Ejemplo: Si cada registro de un empleado mide **4 bytes (ID) + 40 bytes (Nombre) + 8 bytes (Sueldo) = 52 bytes** , el registro número \$N\$ comenzará exactamente en la posición de memoria física:

$$\text{Posición en disco} = (N - 1) \times 52$$

Trabajo Síncrono en Aula: Actividades 2.1 y 2.2

Desarrolla en Eclipse la **Actividad 2.2** del Cuaderno de Aula: cálculo de saltos de puntero y modificación directa de stock mediante `RandomAccessFile` en modo `"rw"`.

3.3. Serialización de Objetos en Java

Guardar un objeto complejo atributo a atributo con tipos primitivos resulta tedioso y propenso a errores. La **Serialización** es el mecanismo nativo de Java para transformar automáticamente un objeto en memoria (junto con todo su grafo de referencias) en una secuencia de bytes para su almacenamiento o transmisión por red.

- **Interfaz** `java.io.Serializable`: Es una interfaz marcadora (no define métodos). Indicar que una clase la implementa autoriza a la Máquina Virtual a inspeccionar y guardar sus campos privados.
- **Palabra clave** `transient`: Aplicada a un atributo, indica que no debe ser serializado (útil para contraseñas en texto claro, conexiones abiertas de red o campos temporales calculados).
- **Identificador** `serialVersionUID`: Un número de versión estático (`private static final long serialVersionUID = 1L;`). Garantiza que la clase que deserializa el objeto sea compatible con la versión con la que se guardó. Si no se declara explícitamente, Java genera uno dinámico y cualquier cambio menor en la clase impedirá recuperar los datos antiguos (lanzando `InvalidClassException`).

La Trampa del Append en ObjectOutputStream

Cada vez que instancias un `ObjectOutputStream`, este escribe automáticamente una **cabecera mágica de 4 bytes** (`0xACED 0005`) al principio del fichero. Si abres un fichero existente en modo append (añadir al final) con el constructor estándar, escribirá una segunda cabecera en mitad del archivo. Al leer posteriormente con `ObjectInputStream`, la lectura fallará catastróficamente al llegar a ese punto lanzando:

```
java.io.StreamCorruptedException: invalid type code: AC
```

Solución profesional: Crear una clase que herede de `ObjectOutputStream` y sobrescriba el método `writeStreamHeader()` dejándolo vacío para omitir la cabecera cuando el fichero ya contenga datos.

Trabajo Síncrono en Aula: Actividades 3.1 y 3.2

Implementa la clase `MiObjectOutputStream` en la **Actividad 3.2** del Cuaderno de Aula para permitir añadir registros a un fichero de clientes serializado sin corromperlo.

3.4. Procesamiento de Ficheros Estructurados XML

El lenguaje XML (*eXtensible Markup Language*) organiza la información en un árbol jerárquico de elementos con etiquetas de apertura y cierre, atributos y texto. En el entorno empresarial de desarrollo Java coexisten tres modelos estándar para su tratamiento:

MODELO XML	TIPO DE API	CONSUMO DE MEMORIA	VENTAJAS PRINCIPALES	CUÁNDO UTILIZARLO
DOM (Document Object Model)	Árbol en memoria	Alto (\$\approx 4\times\$ el tamaño del archivo)	Lectura/escritura bidireccional, navegación libre y soporte XPath.	Documentos pequeños o medianos (< 15 MB) donde se requiere modificar o reordenar nodos.
SAX (Simple API for XML)	Orientado a eventos (Push)	Mínimo (prácticamente cero)	Velocidad máxima, no carga el documento en memoria.	Lectura de grandes ficheros masivos de datos (ej. catálogos de 500 MB o volcados de facturación).
JAXB (Java Architecture for XML Binding)	Mapeo Objeto-XML	Moderado (ligado al POJO)	Convierte clases Java a XML (<i>Marshalling</i>) y viceversa (<i>Unmarshalling</i>) de forma transparente mediante anotaciones.	Servicios web SOAP, pasarelas de pago y consumo de APIs de negocio basadas en XML.

4.1. Nivel Básico: Escritura Atómica con Java NIO.2

Supuesto: Guardar los parámetros de conexión de una aplicación de taller en un archivo local sin necesidad de gestionar bucles ni flujos manuales.

EjemploBásico_NIO2.java

JAVA

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;

public class EjemploBasicoNIO2 {
    public static void main(String[] args) {
        Path rutaConfig = Path.of("config", "app.properties");
        String contenido = "servidor=localhost\npuerto=3306\nusuario=root\n";

        try {
            // Aseguramos la existencia de la carpeta 'config'
            if (Files.notExists(rutaConfig.getParent())) {
                Files.createDirectories(rutaConfig.getParent());
            }

            // Escribe la cadena en una sola operación atómica en UTF-8
            Files.writeString(rutaConfig, contenido,
                StandardOpenOption.CREATE,
                StandardOpenOption.TRUNCATE_EXISTING);

            System.out.println("Configuración guardada correctamente en: " + rutaConfig.toAbsolutePath());
        } catch (IOException e) {
            System.err.println("Error al acceder al disco: " + e.getMessage());
        }
    }
}
```

4.2. Nivel Intermedio: Ingesta y Filtrado de un Fichero CSV de Ventas

Supuesto: Leer un archivo `ventas.csv` con miles de registros de formato `ID;PRODUCTO;PRECIO;UNIDADES`, descartar la cabecera y calcular la facturación total únicamente de los productos de precio superior a 50€.

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.stream.Stream;

public class ProcesadorVentasCSV {
    public static void main(String[] args) {
        Path csvPath = Path.of("datos", "ventas.csv");

        // try-with-resources asegura el cierre del fichero al terminar el stream
        try (Stream lineas = Files.lines(csvPath)) {
            double total = lineas
                .skip(1) // Omitimos la fila de cabecera (ID;PRODUCTO;...)
                .filter(linea -> !linea.isBlank())
                .map(linea -> linea.split(";"))
                .filter(tokens -> tokens.length >= 4)
                .mapToDouble(tokens -> {
                    double precio = Double.parseDouble(tokens[2].trim());
                    int cantidad = Integer.parseInt(tokens[3].trim());
                    return (precio > 50.0) ? (precio * cantidad) : 0.0;
                })
                .sum();

            System.out.printf("Facturación total de artículos premium (>50€): %.2f €\n", total);
        } catch (IOException e) {
            System.err.println("Error I/O procesando el fichero CSV: " + e.getMessage());
        }
    }
}
```

4.3. Nivel Profesional: Actualización In-Situ con RandomAccessFile

Supuesto: Modificar directamente el saldo de una cuenta bancaria almacenada en un archivo de registros binarios de longitud fija sin reescribir el archivo ni cargarlo en memoria.

```
import java.io.RandomAccessFile;
import java.io.IOException;

public class GestorCuentasRAF {
    // Estructura: ID (int: 4 bytes) + Saldo (double: 8 bytes) = 12 bytes por registro
    private static final int TAM_REGISTRO = 12;

    public static boolean actualizarSaldo(String archivoPath, int numeroCuenta, double nuevoSaldo) {
        try (RandomAccessFile raf = new RandomAccessFile(archivoPath, "rw")) {
            long posicionByte = (long) (numeroCuenta - 1) * TAM_REGISTRO;

            // Verificamos si la cuenta existe dentro del límite actual del fichero
            if (posicionByte + TAM_REGISTRO > raf.length()) {
                System.err.println("La cuenta nº " + numeroCuenta + " no existe.");
                return false;
            }

            // Saltamos el ID (4 bytes) y posicionamos el puntero en el saldo
            raf.seek(posicionByte + 4);
            raf.writeDouble(nuevoSaldo);
            return true;
        } catch (IOException e) {
            System.err.println("Fallo de acceso aleatorio: " + e.getMessage());
            return false;
        }
    }
}
```

5 Errores Habituales y Soluciones Técnicas

✗ Error 1: Fugas de Recursos y Bloqueo de Ficheros en el SO

¿Qué ocurre? Abrir un flujo con `new FileReader()` y cerrarlo manualmente al final del método con `fr.close()`. Si ocurre una excepción intermedia, la línea del `close()` nunca llega a ejecutarse.

¿Por qué es grave? En Windows el fichero queda bloqueado por el proceso Java, impidiendo que nadie más lo borre o renombre. En servidores Linux, se agotan los descriptores de fichero del sistema operativo (*Too many open files*).

Solución profesional: Utilizar siempre `try-with-resources`. Cualquier objeto que implemente `AutoCloseable` se cerrará de forma garantizada al salir del bloque, incluso ante fallos graves.

✗ Error 2: Escribir Strings en RAF con Longitud Variable

¿Qué ocurre? Escribir cadenas con `writeUTF(texto)` en un fichero de acceso aleatorio y asumir que el registro siempre ocupará los mismos bytes.

¿Por qué falla? `writeUTF()` escribe 2 bytes con la longitud de la cadena y después los bytes reales del texto. Si guardas "Ana" ocupa 5 bytes, pero si guardas "Alejandro" ocupa 11 bytes. Al hacer `seek((N-1)*tam)`, el puntero aterrizará en mitad de un número provocando lecturas absurdas.

Solución profesional: Fijar siempre una longitud máxima de caracteres (rellenando con espacios en blanco o usando `StringBuffer.setLength()`).

✗ Error 3: Procesar XML Masivos con DOM provocando OutOfMemoryError

¿Qué ocurre? Intentar parsear un fichero XML de 200 MB descargado de la Agencia Tributaria utilizando `DocumentBuilder.parse()`.

¿Por qué falla? La especificación DOM crea en memoria un objeto Java completo para cada nodo, atributo y fragmento de texto. Un archivo XML de 200 MB requiere típicamente entre 800 MB y 1.2 GB de memoria RAM libre, colapsando la Máquina Virtual.

Solución profesional: Usar siempre un parser de flujo como **SAX** o **StAX** para ficheros de gran tamaño.

6 Buenas Prácticas de la Industria

Niveles de Cumplimiento Técnico

En el desarrollo de software corporativo distinguimos tres categorías claras de diseño en persistencia física:

- **Obligatorio (Para que el código sea admitido en producción):**
 - Especificar siempre el juego de caracteres: `StandardCharsets.UTF_8`.
 - Envolver todos los flujos de I/O en bloques `try-with-resources`.
 - Utilizar `Path.of()` o constructores que usen separadores agnósticos (`File.separator`); nunca usar barras invertidas de Windows a fuego como `"C:\\datos\\fichero.txt"`.
- **Altamente Recomendable (Calidad y Mantenibilidad):**
 - Bufferizar siempre las operaciones de entrada/salida de bajo nivel.
 - Definir siempre de forma explícita el `private static final long serialVersionUID` en todas las clases serializables.
 - Separar las rutas físicas del código de negocio mediante ficheros de propiedades o variables de entorno.
- **Estándar de la Industria (Arquitectura):**
 - No bloquear el hilo principal de la aplicación durante operaciones I/O intensivas (usar I/O asíncrono o procesamiento reactivo con NIO.2).

7 Resumen Clave de la Unidad

Conceptos esenciales para repasar antes de cualquier prueba de evaluación o entrega de proyecto:

- **Streams:** Flujos de datos unidireccionales. Se dividen en flujos de Bytes (`InputStream` / `OutputStream`) y flujos de Caracteres (`Reader` / `Writer`).
- **UTF-8:** Es el juego de caracteres universal que debe especificarse siempre para evitar corrupción de tildes o caracteres especiales al migrar entre sistemas operativos.
- **Java NIO.2:** API moderna basada en `Path` y métodos de utilidad en `Files` que simplifica drásticamente el código y ofrece streams funcionales con `Files.lines()` .
- **RandomAccessFile:** Permite leer y modificar datos saltando libremente por el fichero mediante su puntero con `seek()` ; requiere registros de longitud constante.
- **Serialización:** Almacenamiento nativo de objetos mediante `ObjectOutputStream` . Exige implementar `Serializable` , definir `serialVersionUID` y usar `transient` para omitir datos confidenciales.
- **XML:** Utiliza **DOM** para ficheros pequeños que necesitan navegación completa en árbol, **SAX** para ingesta rápida de ficheros gigantes con consumo nulo de memoria, y **JAXB** para mapear POJOs a XML mediante anotaciones.

8 Conceptos que Debes Dominar

Al concluir el estudio de esta unidad de trabajo, debes ser capaz de:

- ✓ **Comprender** la diferencia fundamental entre flujos de bytes y flujos de caracteres.
- ✓ **Justificar** por qué es indispensable usar codificación UTF-8 en cualquier aplicación Java moderna.
- ✓ **Saber utilizar** `BufferedReader` , `BufferedWriter` y `PrintWriter` para procesar texto plano.
- ✓ **Dominar** la clase `Files` de NIO.2 para lectura y escritura atómica y filtrado funcional de ficheros masivos.
- ✓ **Ser capaz de calcular** la posición exacta en bytes de un registro en `RandomAccessFile` y actualizar su contenido sin corromper el resto.
- ✓ **Implementar** correctamente la serialización de clases con `serialVersionUID` y campos `transient` .
- ✓ **Resolver** el problema de la doble cabecera en ficheros serializados mediante una subclase especializada de `ObjectOutputStream` .
- ✓ **Identificar** cuándo aplicar un parser DOM, un parser SAX o mapeo JAXB según el volumen de datos y los requisitos de memoria del sistema.

Glosario de Términos Clave

- **Stream (Flujo):** Secuencia ordenada de bytes o caracteres transmitida de forma continua entre un emisor y un receptor.
- **Charset (Juego de Caracteres):** Mapeo binario estandarizado que asigna un valor numérico específico a cada carácter alfanumérico o símbolo gráfico.
- **Buffer:** Zona temporal de memoria de alta velocidad que almacena datos intermedios para reducir el número de accesos costosos a disco.
- **Puntero de Fichero:** Marcador numérico en `RandomAccessFile` que indica el índice de byte exacto en el que tendrá lugar la siguiente operación de lectura o escritura.
- **transient:** Modificador de campo en Java que excluye un atributo del proceso de serialización automática.
- **serialVersionUID:** Identificador de 64 bits que verifica que la clase emisora y receptora de un objeto serializado compartan la misma estructura de campos.
- **Marshalling / Unmarshalling:** Proceso de transformar un grafo de objetos Java en una estructura de documento XML (Marshalling) y el proceso inverso (Unmarshalling).

Banco de Preguntas de Repaso (Sin Respuesta Inmediata)

Utiliza estas 10 preguntas para verificar tu nivel de asimilación antes del examen práctico o de la prueba diagnóstica:

Pregunta 1: Explica la diferencia entre usar `FileReader` directamente y envolverlo en un `BufferedReader`. ¿Qué ventajas aporta a nivel de rendimiento y qué método adicional proporciona?

Pregunta 2: ¿Por qué un fichero de texto guardado en un equipo Windows puede verse con caracteres extraños al abrirlo en un servidor Linux si no se especifica el Charset? ¿Cómo se previene en Java?

Pregunta 3: Describe la diferencia de comportamiento entre los métodos `Files.readAllLines()` y `Files.lines()` de Java NIO.2. ¿Cuál utilizarías para un fichero de 5 GB de logs y por qué?

Pregunta 4: En un fichero binario gestionado con `RandomAccessFile` donde cada registro de artículo mide exactamente 32 bytes, ¿qué instrucción Java exacta posiciona el puntero en el inicio del registro número 8?

Pregunta 5: Si modificas el valor de un atributo en un registro usando `RandomAccessFile`, ¿se borran los registros que vienen después en el fichero? Justifica cómo gestiona el sistema de ficheros esta operación.

Pregunta 6: ¿Qué utilidad tiene la palabra clave `transient` en una clase que implementa `Serializable`? Pon un ejemplo de un atributo en una aplicación bancaria donde sea indispensable utilizarla.

Pregunta 7: ¿Qué excepción se produce al intentar añadir objetos al final de un fichero ya existente usando `new ObjectOutputStream(new FileOutputStream(path, true))`? Explica la causa física a nivel de bytes.

Pregunta 8: ¿Cómo soluciona la industria el problema de añadir objetos a un fichero serializado en modo append sin que se corrompa el archivo?

Pregunta 9: Un cliente te solicita una aplicación para extraer datos de facturación de ficheros XML de 1.5 GB que le envían diariamente sus proveedores. ¿Qué tecnología de procesamiento XML seleccionarías (DOM o SAX) y por qué motivo técnico?

Pregunta 10: ¿Qué anotación de JAXB es obligatoria en la clase raíz para que el **Marshaller** pueda serializar un objeto a un documento XML válido?